

Simple Wave Function Collapse

Grid

A `Grid` represents the domain, i.e. the values for each pixel. It is defined as a cartesian domain, `rows` by `cols`.

Tile

Tiles are square subgrid of size `N` (which is a template parameter). They define a local valid combination of the values of a pixel.

Algorithm

A good presentation of the algorithm can be found [here](#).

Overview

```
Inputs: a sample grid S, a problem grid G, a tile size N
1. Generate the list L of valid tiles from S
2. For each tile t of L, compute the list valid(t, offset) of tiles that can
   overlap with a given offset
3. For each pixel of G, compute its entropy, i.e. the number of possible
   solutions according to the valid rules previously computed
4. Select the pixel of the lowest entropy and choose randomly a valid
   solution. If none, it's a fail.
5. Update entropy of the neighbors
6. Loop to 4
7. Return the computed configuration
```

1. Valid tiles

From the sample grid S , we generate the list of valid tiles L .

A tile t is valid if it is a subgrid of size $N \times N$ of S .

The frequency of a tile t is the number of times it appears in S .

For example, given the following sample grid S of size 5×5 and with coefficients in $\{0, 1\}$:

```
1 0 1 1 1
1 0 1 1 1
```

```
0 0 1 1 1
0 1 1 1 1
0 0 0 0 0
```

The valid tiles of size 2×2 are:

```
1 0
1 0
```

it is found 1 time.

```
1 0
0 0
```

it is found 1 time.

```
0 0
0 1
```

it is found 1 time.

```
0 1
0 0
```

it is found 1 time.

```
0 1
0 1
```

it is found 2 times.

```
0 1
1 1
```

it is found 1 time.

```
1 1
0 0
```

it is found 3 times.

```
1 1
1 1
```

it is found 6 time.

Note that more complex definitions of tiles are possible, for example allowing geometrical symmetries (rotations, reflections).

2. Overlapping tiles

For each tile t of L , we compute the list of valid tiles that can overlap with a given overlapping offset.

For example, given the following tile t :

```
0 1
0 1
```

We want to characterize its neighbors:

```
+-----> x
|  * * * *
|  * 0 1 *
|  * 0 1 *
|  * * * *
v
y
```

The valid tiles that can overlap with an offset of 1, 0 are:

```
1 0
1 0
```

and

```
1 0
0 0
```

Because

```
1 [0] 1
```

```
1 [0] 0
```

and

```
1 [0] 1
0 [0] 0
```

do not conflict.

For offset 0, 1, the only valid tile is:

```
1 0
[1] [0]
```

For offset 1, 1, valid tiles have [1] in their upper left corner.

3. Entropy

For each pixel of , we compute its entropy, i.e. the number of possible solutions according to the rules previously computed.

This computation is done in two steps:

1. Identify which tiles can contain the pixel
2. Identify the subset of valid tiles that can overlap with the pixel and its neighbors. The entropy is the number of valid tiles.

Using the previous example,

```
+-----> x
|  *  *  *  *
|  (*) 0 1 *
|  [*] 0 1 *
|  *  *  *  *
v
y
```

- (*) has an entropy of 1, because the only two valid tiles that can overlap with the already placed tiles have a 1 in this position.
- [*] has an entropy of 2, because 0 or 1 are valid

4. Resolution

Select the pixel of the lowest entropy and choose randomly a tile from the set of valid tiles. If there is none, it's a fail. If there are several, choose the value according to the frequency of the tiles.

Continuing the previous example, we can set the pixel (*) to 1 .

5. Update entropy

Update the entropy of the neighbors of the pixel .

Continuing the previous example, if we can set the pixel (*) to 1 , we have:

```
+-----> x
| [*] [*] [*] * *
| [*] (1) 0 1 *
| [*] [*] 0 1 *
| *  *  *  * *
v
y
```

where [*] marks the pixels for which we have to update the entropy measurement.

Work to do

1. Implement a serial version of the algorithm, using simple grids of 0 and 1 .
2. Implement a parallel version of the algorithm, using Kokkos OR OpenMP.

In particular, for OpenMP, you should use the explicit task API.

3. Implement a version of the algorithm that uses a more complex grid, with more than two values.

Experiment and comment on the performance of the algorithm.

In particular, you should comment on the performance of the parallel version of the algorithm and its scalability.

4. Project Report

After a short description of the project and your understanding, you will explain the strategy you employed to implement the project.

In particular, issues that you might have met during the project development.

At the end, we also expect a detailed performance analysis while varying input dataset, configuration etc.

5. Presentation

We will organize a project presentation on **12th May, 2026**.

The slot will last 25min, including questions from the jury.

Projects and reports must be submitted **no later than 11.59pm on 05 May 2026**.

Evaluation

You will be evaluated on:

- the project,
- the report,
- the presentation.